CYBER SECURITY CAPSTONE

Network Traffic Analyzer

Project Report

Name: Shreeja Ubhare
Submission date: October 23, 2026

Abstract

This project develops a local network traffic analyzer for security monitoring. It captures a bounded sample of packets visible to an authorized network interface, records header metadata rather than payload contents, summarizes traffic with Pandas, and presents results in a Streamlit dashboard. The system visualizes protocol distribution, packet counts over time, source activity, and incoming or outgoing direction. It stores capture sessions and packet metadata in MySQL and exports both a traffic/security summary and packet rows as downloadable files. Explainable threshold rules identify patterns such as high packet counts, many destination ports, packet-rate spikes, repeated initial TCP connection requests, high byte volume, and non-public source addresses absent from a trusted list. Testing included a 20-packet live capture and synthetic traffic; a 476-row synthetic sample was saved and reloaded with its direction counts preserved. The alerts are heuristic review leads, not proof of malicious activity, and the application is not a replacement for a production intrusion-detection system.

Keywords: network monitoring, packet metadata, Scapy, Streamlit, MySQL, anomaly heuristics

Contents

1 Introduction **1**
- 1.1 Problem statement . . . 1
- 1.2 Objectives . . . 1

2 Background and concepts **1**
- 2.1 Packets and metadata . . . 1
- 2.2 IP addresses, protocols, and ports . . . 1
- 2.3 Monitoring and anomalies . . . 1

3 Requirements and technology **2**

4 System design **2**
- 4.1 Packet capture . . . 2
- 4.2 Traffic direction . . . 2
- 4.3 Database model . . . 2

5 Implementation **3**
- 5.1 Dashboard and monitoring . . . 3
- 5.2 Detection rules . . . 3
- 5.3 Persistence and reports . . . 3

6 Results and validation **3**

7 Limitations and safe-use boundary **4**

8 Future improvements **4**

9 Conclusion **5**

1 Introduction

Network communication is divided into packets that carry addressing and protocol information alongside application data. Security and network-administration teams use traffic observations to understand communication patterns and identify behavior that may deserve review. This capstone implements a small, local analyzer to demonstrate that workflow using Python.

1.1 Problem statement

Raw packet observations are difficult to summarize by inspection. A learner needs a safe way to capture a limited amount of authorized traffic, organize packet metadata, view aggregate patterns, and examine simple security alerts.

1.2 Objectives

The project aims to:

- capture a bounded set of IPv4 or IPv6 packet metadata from an authorized interface;
- summarize packet counts, sizes, protocols, source addresses, and traffic direction;
- apply transparent, configurable heuristics and show why a finding was raised;
- display traffic summaries and charts in a local dashboard;
- persist capture sessions and packet metadata in MySQL; and
- export a human-readable summary and a packet-level CSV file.

2 Background and concepts

2.1 Packets and metadata

A packet contains headers used for delivery and protocol processing, and may contain a payload carrying application data. This application records a limited metadata row: timestamp, source and destination IP addresses, protocol, available transport ports, packet length, direction, and TCP flags when available. Payload contents are not stored.

2.2 IP addresses, protocols, and ports

An IP address identifies an interface at the network layer. TCP and UDP are transport protocols commonly used by applications. TCP uses connection-management flags and provides an ordered byte stream; UDP carries datagrams without TCP's connection setup. Ports help identify application endpoints, but a port number alone does not prove which application generated traffic.

2.3 Monitoring and anomalies

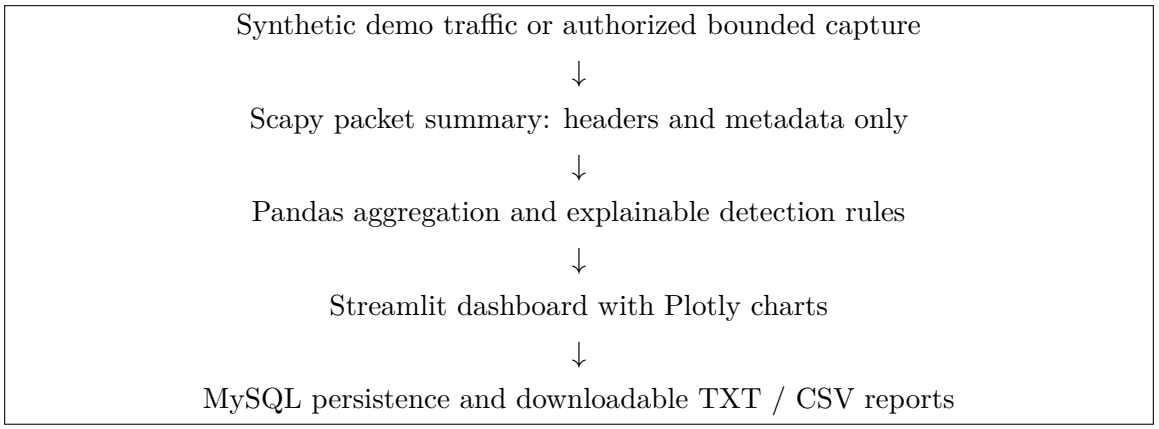
Monitoring aggregates observations, for example by protocol, source address, or time interval. An anomaly is a deviation from an expected baseline, not necessarily malicious activity. This project uses simple threshold heuristics because they are understandable and suitable for demonstration. Their interpretation requires context about the device, capture duration, and normal workload.

3 Requirements and technology

Component	Use in the project
Python	Application logic and packet-summary processing.
Scapy and Npcap	Bounded live packet capture on Windows interfaces.
Pandas	Tabular storage in memory, grouping, counts, and time-based summaries.
Streamlit	Local interactive dashboard and capture controls.
Plotly	Protocol, timeline, source-activity, and direction charts.
MySQL 8.4 and Connector/Python	Persistent capture and packet-metadata storage.

4 System design

The application follows a simple processing flow:



4.1 Packet capture

Scapy observes packets available to the selected local interface. Capture is bounded by a maximum packet count and a short timeout. Visibility depends on the interface, operating system, Npcap, permissions, and network configuration. The program summarizes IP packets and does not retain their payloads. Scapy's official documentation describes packet capture and dissection capabilities [1].

4.2 Traffic direction

Direction is inferred by comparing packet source and destination addresses with local IP addresses supplied for the monitored device. A packet from a known local address is marked outgoing; a packet to a known local address is marked incoming. If the addresses do not match the configured list, direction is marked unknown. This is a practical heuristic, not a guarantee about the physical path of a packet.

4.3 Database model

The MySQL schema uses two related tables. `captures` stores one row per capture session, including its identifier, creation time, source label, and packet count. `packets` stores packet metadata and a `capture_id` foreign key. One capture session can therefore have many associated packet rows. MySQL Connector/Python provides the connection between the Python application and MySQL Server [4].

5 Implementation

5.1 Dashboard and monitoring

The dashboard displays total packets, captured bytes, distinct source IP addresses, and protocol types. Charts show protocol distribution, packet counts in time buckets, top source activity, and packet counts by direction. Captured bytes are the sum of recorded packet lengths; they are a total for the selected capture, not a rate such as bytes per second. Streamlit provides the interactive application components and layout [2]; Pandas supports tabular grouping and data processing [3].

5.2 Detection rules

The dashboard applies explainable rules to the selected traffic:

1. **High packet count:** flags a source with at least the configured packet count.
2. **High byte volume:** flags a source whose summed packet lengths exceed the configured byte threshold.
3. **Many destination ports:** flags a source contacting many distinct destination ports.
4. **High packets per minute:** flags a source whose count in a one-minute bucket reaches the configured rate threshold.
5. **Repeated TCP connection requests:** counts initial SYN packets to the same destination address and port.
6. **Unrecognized non-public source IP:** flags a non-public source address that is absent from the user's trusted-IP list.

These rules create review leads. They can produce false positives, and an absence of alerts does not establish that traffic is benign.

5.3 Persistence and reports

The user can save packet metadata as a capture session in MySQL and reload it later. The dashboard also offers a text report containing summary metrics, protocol and direction counts, and security findings, plus a CSV export containing packet-level rows. Missing optional metadata is stored as SQL `NULL`; the database does not store packet payload contents.

6 Results and validation

The application was exercised with both live and synthetic data:

- A Scapy live-capture demonstration summarized 20 IP packets from the local device's visible traffic.
- A selected saved capture displayed 98 packets, 120,037 captured bytes, 10 distinct source addresses, and two protocols; its protocol chart showed 95 TCP packets and 3 UDP packets.
- The synthetic dashboard sample contained 476 rows: 450 generated baseline rows, 20 scan-like demonstration rows, and 6 repeated-SYN demonstration rows. Its direction chart showed 364 outgoing and 112 incoming rows.
- A synthetic capture was saved to MySQL and reloaded. The 476-row total and direction counts remained available after reload.
- Threshold behavior was demonstrated: a source with 29 packets in a one-minute bucket triggered a threshold of 20; raising the threshold to 30 removed that finding.

- Adding the synthetic source address `192.168.1.25` to the trusted-IP list removed its unrecognized-address alert.

7 Limitations and safe-use boundary

- The application captures only traffic visible to the selected local interface; it does not monitor an entire switched network.
- Live capture may require Npcap and elevated permissions on Windows, and can be affected by VPNs, firewalls, or packet loss.
- Captures are short and bounded; the dashboard is not a continuous real-time monitoring service.
- Threshold rules are simple heuristics without a learned baseline. Legitimate updates or busy services can trigger alerts, and subtle threats can be missed.
- Direction depends on the configured local-address list. Older records or synthetic data without direction metadata may appear as unknown.
- The unrecognized-address rule is an indicator for human review, not a device inventory or attribution system.
- Only packet metadata is retained. The analyzer does not inspect payload contents, perform network scans, attempt logins, or access remote systems.

Use live capture only on a device or network owned by the user or explicitly authorized for monitoring.

8 Future improvements

Possible extensions include calculating bytes per second from accurate timestamp windows, tracking active connections from operating-system socket tables, adding time-range filters, recording alerts as database events, supporting configurable device inventories, and improving the test suite. Any extension should retain the project's authorized, metadata-focused scope.

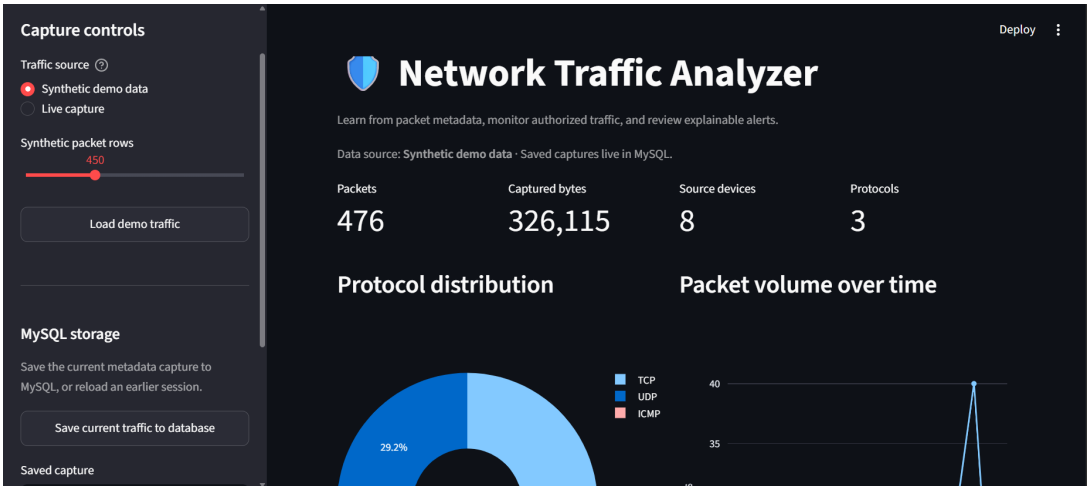


Figure 1: Network Traffic Analyzer dashboard showing traffic metrics, visualizations, and activity alerts.

9 Conclusion

The Network Traffic Analyzer demonstrates a complete introductory workflow: capture or generate packet observations, summarize metadata, apply transparent rules, visualize activity, persist records in MySQL, and export reports. The working prototype provides a foundation for learning network monitoring while making its safety limits and uncertainty explicit.

References

[1] Scapy Project, *Scapy Documentation: Usage*. https://scapy.readthedocs.io/en/stable/usage.html

[2] Streamlit, *API Reference*. https://docs.streamlit.io/develop/api-reference

[3] The pandas development team, *pandas User Guide*. https://pandas.pydata.org/docs/user_guide/

[4] Oracle, *MySQL Connector/Python Developer Guide*. https://dev.mysql.com/doc/connector-python/en/

[5] Python Software Foundation, *ipaddress — IPv4/IPv6 manipulation*. https://docs.python.org/3/library/ipaddress.html